Program:

Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
print("All libraries imported successfully!")
```

output

```
All libraries imported successfully!
```

Upload the CSV file

```
uploaded = files.upload()
filename = list(uploaded.keys())[0]
print("Uploaded file:", filename)
```

output

```
house-price-data.csv(text/csv) - 12740 bytes, 100% done
Saving house-price-data.csv to house-price-data.csv
Uploaded file: house-price-data.csv
```

Load and display the dataset

```
df = pd.read_csv(filename)
print("Dataset loaded successfully!")
print()
print("First 5 rows of the dataset:")
display(df.head())
```

output

Dataset loaded successfully!

First 5 rows of the dataset:

	Area_SqFt	Bedrooms	Bathrooms	Age_Years	Garage	Distance_KM	Price
0	1983	4	2	22	3	9.3	332988
1	1176	2	1	41	3	23.8	164619
2	2250	3	2	14	0	19.3	343393
3	2364	3	2	12	1	27.9	313474
4	629	1	1	17	3	29.4	104963

Check dataset information

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
print("\nColumn names:")
for column in df.columns:
    print("-", column)
```

output

Number of rows: 500

Number of columns: 7

Column names:

- Area_SqFt
- Bedrooms
- Bathrooms
- Age_Years
- Garage
- Distance_KM
- Price

Check missing values

```
print("Missing values in each column:\n")
print(df.isnull().sum())
```

output

Missing values in each column:

Area_SqFt 0

Bedrooms 0

Bathrooms 0

Age_Years 0

Garage 0

Distance_KM 0

Price 0

dtype: int64

Data Cleaning

```
numeric_columns = [
    "Area_SqFt",
    "Bedrooms",
    "Bathrooms",
    "Age_Years",
    "Garage",
    "Distance_KM",
    "Price"
]
for column in numeric_columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")
df = df.dropna()
print("Data cleaning completed successfully!")
print("Rows after cleaning:", len(df))
```

output

Data cleaning completed successfully!

Rows after cleaning: 500

Statistical Analysis

```
print("STATISTICAL SUMMARY")
```

print("==================")

display(df[numeric_columns].describe().round(2))

output

STATISTICAL SUMMARY
==================

	Area_SqFt	Bedrooms	Bathrooms	Age_Years	Garage	Distance_KM	Price
count	500.0	500.0	500.0	500.0	500.0	500.0	500.0
mean	1794.5	2.75	1.77	25.01	1.47	15.26	261832.76
std	570.6	1.13	0.82	14.85	1.1	8.47	91969.44
min	600.0	1.0	1.0	0.0	0.0	1.1	30485.0
25%	1400.0	2.0	1.0	12.0	0.0	8.15	198752.5
50%	1802.0	3.0	2.0	24.0	2.0	15.35	261074.0
75%	2152.75	4.0	2.0	39.0	2.0	22.8	322122.0
max	3548.0	6.0	5.0	50.0	3.0	30.0	538519.0

Average Price by Number of Bedrooms

bedroom_avg = df.groupby("Bedrooms")["Price"].mean()

print("AVERAGE PRICE BY BEDROOMS")

print("========================")

for beds, value in bedroom_avg.items():

    print(beds, "BHK :", round(value, 2))

output

AVERAGE PRICE BY BEDROOMS

========================

1 BHK : 158631.48

2 BHK : 209400.72

3 BHK : 279576.57

4 BHK : 332059.06

5 BHK : 428961.2

6 BHK : 483652.0

Bar Graph

plt.figure(figsize=(9, 6))

plt.bar(bedroom_avg.index.astype(str), bedroom_avg.values)

plt.title("Average House Price by Bedrooms")

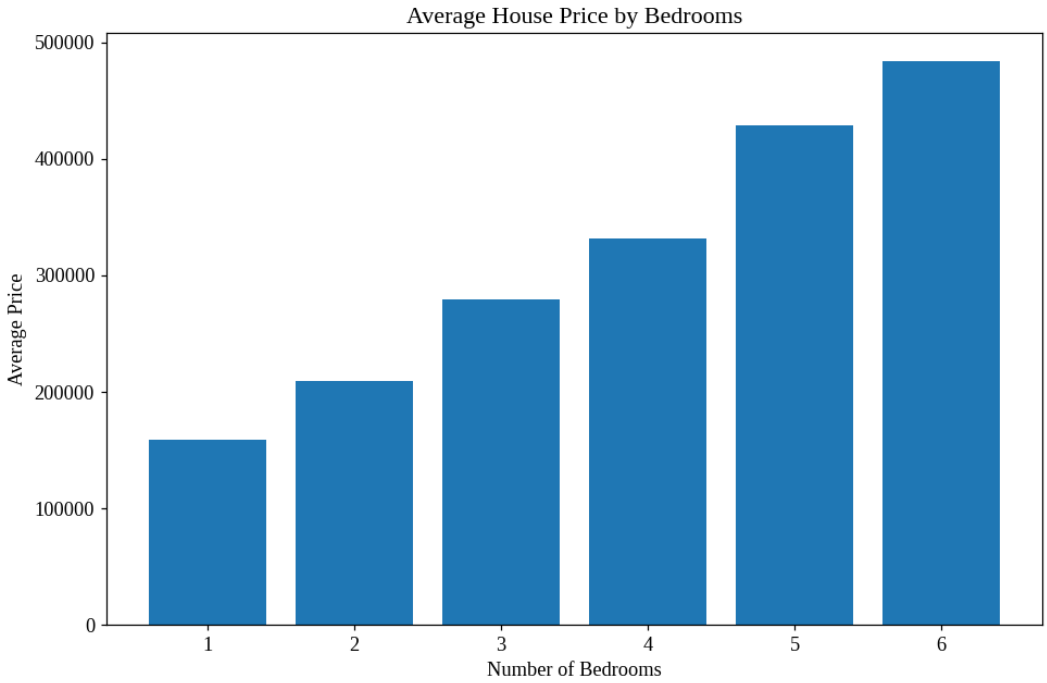
plt.xlabel("Number of Bedrooms")

plt.ylabel("Average Price")

plt.tight_layout()

plt.show()

output



Area vs Price Trend

plt.figure(figsize=(12, 6))

plt.scatter(df["Area_SqFt"], df["Price"], alpha=0.6)

plt.title("House Area vs Price")

plt.xlabel("Area (Sq Ft)")

plt.ylabel("Price")

plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()

plt.show()

output



Prepare data for Machine learning

```
X = df[
    [
        "Area_SqFt",
        "Bedrooms",
        "Bathrooms",
        "Age_Years",
        "Garage",
        "Distance_KM"
    ]]
y = df["Price"]
print("Input Features (X):")
print(X.columns.tolist())
print("\nTarget Variable (y):")
print("Price")
```

output

```
Input Features (X):
['Area_SqFt', 'Bedrooms', 'Bathrooms', 'Age_Years', 'Garage', 'Distance_KM']

Target Variable (y):
Price
```

Split training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("Data splitting completed!")
print()
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

output

```
Data splitting completed!

Training samples: 400
Testing samples: 100
```

Train the machine learning model

```
model = LinearRegression()
model.fit(X_train, y_train)
print("Linear Regression model trained successfully!")
```

output

```
Linear Regression model trained successfully!
```

Make Predictions

```
y_pred = model.predict(X_test)
print("Predictions generated successfully!")
print("\nFirst 10 predictions:")
for i in range(min(10, len(y_pred))):
    print(
        "Actual:",
        round(y_test.iloc[i], 2),
```

```
        "| Predicted:",
        round(y_pred[i], 2)
    )
```

output

Predictions generated successfully!

First 10 predictions:

Actual: 272879 | Predicted: 289482.62

Actual: 303013 | Predicted: 263456.28

Actual: 233174 | Predicted: 266361.98

Actual: 379572 | Predicted: 381035.95

Actual: 167818 | Predicted: 189375.29

Actual: 292160 | Predicted: 282069.54

Actual: 199538 | Predicted: 128681.55

Actual: 292169 | Predicted: 335095.7

Actual: 357276 | Predicted: 347846.72

Actual: 160546 | Predicted: 205031.61

Evaluate the model

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("MODEL PERFORMANCE")
print("=================")
print("MAE :", round(mae, 2))
print("MSE :", round(mse, 2))
print("RMSE :", round(rmse, 2))
print("R² :", round(r2, 4))
```

output

MODEL PERFORMANCE

==========

MAE : 23286.64

MSE : 790395375.18

RMSE : 28113.97

R^2 : 0.9022

Actual vs Predicted table

```
results = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": np.round(y_pred, 2)
})
print("ACTUAL VS PREDICTED")
print("====================")
display(results.head(15))
```

output

ACTUAL VS PREDICTED

==========

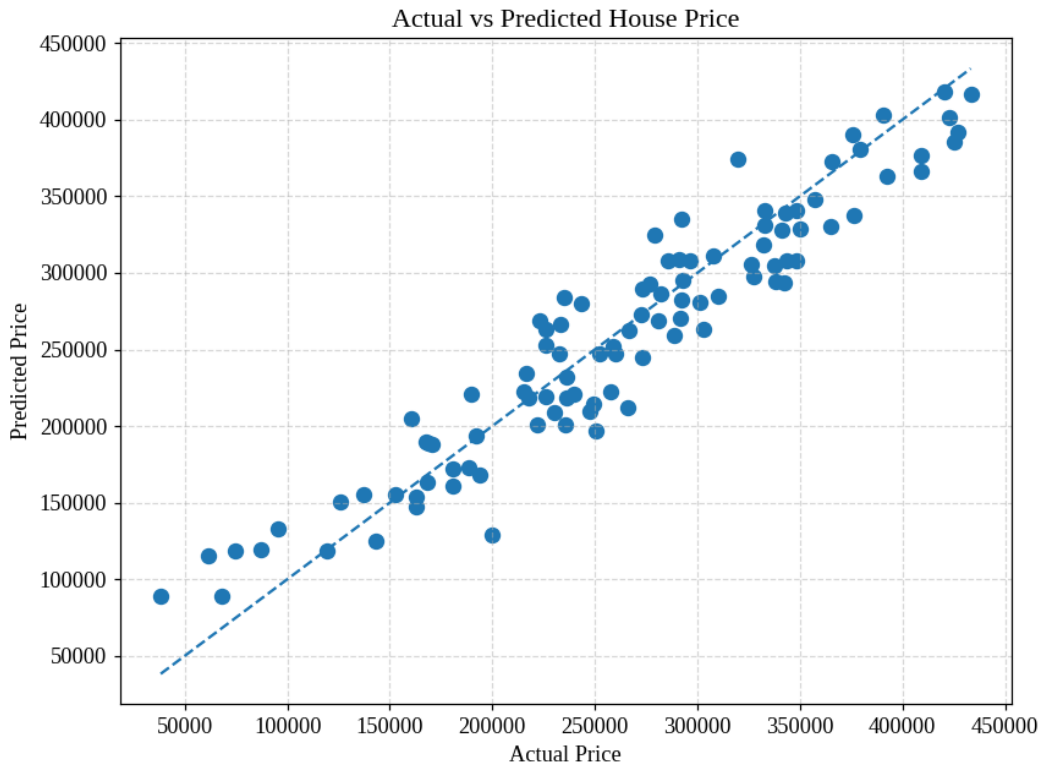
	Actual Price	Predicted Price
0	272879	289482.62
1	303013	263456.28
2	233174	266361.98
3	379572	381035.95
4	167818	189375.29
5	292160	282069.54
6	199538	128681.55
7	292169	335095.7
8	357276	347846.72
9	160546	205031.61
10	266226	212440.62
11	193992	167949.32
12	292730	295519.55
13	250465	196644.89
14	343144	338858.13

Actual vs predicted graph

```
plt.figure(figsize=(8, 6))
plt.scatter(
    y_test,
    y_pred,
    s=60
)
minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())
plt.plot(
    [minimum, maximum],
    [minimum, maximum],
```

```
    linestyle="--"
)
plt.title("Actual vs Predicted House Price")
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

output



Test the model with new house data

```
new_data = pd.DataFrame({
    "Area_SqFt": [2000],
    "Bedrooms": [3],
    "Bathrooms": [2],
    "Age_Years": [10],
    "Garage": [1],
```

```
    "Distance_KM": [8]
})
prediction = model.predict(new_data)
print("NEW HOUSE DATA")
print("==============")
print("Area :", 2000, "Sq Ft")
print("Bedrooms :", 3)
print("Bathrooms :", 2)
print("Age :", 10, "years")
print("Garage :", 1)
print("Distance :", 8, "KM")
print()
print("Predicted House Price:",
      round(prediction[0], 2))
```

output

```
NEW HOUSE DATA
==============
Area : 2000 Sq Ft
Bedrooms : 3
Bathrooms : 2
Age : 10 years
Garage : 1
Distance : 8 KM

Predicted House Price: 321820.97
```

Save the prediction results

```
results.to_csv(
    "house_price_predictions.csv",
    index=False
)
```

```
print("Results saved successfully!")
print("File name: house_price_predictions.csv")
```

output

```
Results saved successfully!
File name: house_price_predictions.csv
```